

Quick Sort

Page _____

Algorithm:

Step 1: Start

Step 2: Read array a , low & high

Step 3: ~~key~~ key = $a[\text{low}]$, $i = \text{low} + 1$, $j = \text{high}$

Step 4: while $a[i] \leq \text{key}$ if $i < \text{high}$

$i++$

while $a[j] \geq \text{key}$

$j--$

if $(i > j)$

swap $(a[i], a[j])$

else

swap $(a[j], a[\text{low}])$

return j

Step 5: if $(\text{low} < \text{high})$

~~apptp~~ quick sort $(a, \text{low}, j-1)$

quicksort $(a, j+1, \text{high})$

Step 6: Exit

Code.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int partition(int a[], int low, int high)
```

```
{  
    int i, j, key;
```

```
j = high;  
key = a[low];  
while (i < j)
```

```
{  
    while (a[i] <= key && i < high)
```

```
        i++;
```

```
    while (a[j] > key)
```

```
        j--;
```

```
    if (i < j)
```

```
    {  
        int temp = a[i];
```

```
        a[i] = a[j];
```

```
        a[j] = temp;
```

```
    }
```

```
    else
```

```
    {
```

```
        int temp = a[low];
```

```
        a[low] = a[j];
```

```
        a[j] = temp;
```

```
        return j;
```

```
    }
```

```
}
```

```
}
```

```
void quicksort (int a[], int low, int high)
```

```
{
```

```
    int j;
```

```
    if (low < high)
```

```
    {
```

```
        j = partition (a, low, high);
```

```
        quicksort (a, low, j-1);
```

```
        quicksort (a, j+1, high);
```

```
    }
```

```

int main()
{
    int n, a[100];
    printf("Enter the no. of elements in array: ");
    scanf("%d", &n);
    printf("Enter the elements of array: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    clock_t start, end;
    start = clock();
    quickSort(a, 0, n-1);
    end = clock();

    printf("Sorted array: \n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t", a[i]);
    }

    double time = (double) (end - start) / CLOCKS_PER_SEC;
    printf("Execution time : %f", time);
}

```

Output

Enter the ^{no. of} elements in array: 10
 Enter the elements of the array: 5 99 64
 3 16 78 20 11 53 61

Sorted array:

3 5 11 16 20 53 61 64 78 99

Execution time: 0.000000